붙임 **2.** 아이디어 요약서 **(Sol-AIM** 결과물 요약서**)**

항목	내용
출품작 제목	C.A.T.T.U.S (Coding Agent That Truly Useful and Safe)
팀명 / 대표자	Le Huu Duy (가 ,AI&Big Data 학과)
가 부문	학생
분야	Agentic AI
결과물 유형	AI 활용 시스템 (학습 도구·분석 대시보드)

② 배경 · 문제 정의

(1) 까 있는가 문제가

프로그래밍을 공부하는 학생들은 Cursor, GitHub Copilot, Claude Code, ChatGPT 같은 AI 코딩 도구를 매우 능숙하게 사용합니다. 그러나 교육 현장에서는 AI가 만든 코드를 이해하지 않은 채 그대로 제출하거나, AI가 잘못 짠 코드를 검증 없이 커밋하는 학습 부작용이 자주 나타납니다. 학생은 어느 줄이 위험한지, 정말 통과하는지 판단하기 어렵고, 실패하면 다시 AI에 "고쳐줘"를 반복하며 학습은 거의 일어나지 않습니다.

(2) 가 중기요존한가해법의 한계

학생은 코드 자체를 이해하는 능력을 잃고, 같은 습관이 산학·실무에서 보안·품질 사고로 이어집니다.
Cursor·Copilot·Claude Code는 생산성 도구이지 학습·가 도구가다. "왜 위험한지" "정말 통과하는지"를 학생 눈높이에서 설명·검증하도록 설계되지 않았습니다.
기존 LMS·자동 채점은 정답 유무만 알려줄 뿐, AI가 만든 코드의 근거·위험도·승인 절차를 다루지 못합니다.

(3) 왜 지금 **AI**가 하는가 해결해야

AI 코딩 도구는 이미 학생 워크플로의 일부입니다. 도구를 금지하는 대신 그 위에 안전·학습 계층을 얹는 방향이 더 현실적이며, 본 출품작은 그 계층을 학생에게 직접 제공합니다.

---

③ 접근 방법

(1) 한 줄 요약

C.A.T.T.U.S는 Cursor/Copilot/Claude Code/ChatGPT의 대체재가 아니라 동반자입니다. 학생이 그런 도구로 만든 코드를 가 ⑪ 시스템이 ① 설명하고, ② 위험을 검토하고, ③ 벤치마크로 검증하고, ④ 사람이 직접 승인한 뒤에만 실제 코드에 반영합니다.

(2) 전체 흐름 **— "**이해 → 검토 → 검증 → 승인**"**

[질문/코드] → [1] Goal Brief → [2] Indexer+Control Engine
|
[3] AI 협업(Gemini→DeepSeek/Ollama 리뷰)
|
[4] Patch Safety Gate → [5] 임시 클론 검증
|
[6] 인간 승인(Mayor Mode) → [7] Memory(Lesson)

(3) 핵심 아이디어 **— "AI**는 보조**,** 결정은 사람**"**

각 단계에서 AI는 후보·근거·위험도를 만들어 제시하고, 적용**/**승인 결정은 항상 학생**(**또는 지도자**)**이 합니다. 메모리(lesson)도 사람이 "Promote" 버튼을 눌러야만 활성화됩니다.

(4) AI 활용의 큰 그림

다중 모델 협업으로 Gemini(초안) → DeepSeek/Ollama(리뷰)를 거쳐 단일 모델 환각을 줄이고, Tree-sitter 기반 그래프로 프로젝트를 이해하며 과거 실패 이력이 있는 파일을 우선 노출합니다(universal node weight + lesson penalty). 모든 실험은 .tmp/ 복제본에서 먼저 돌려 실 저장소를 보호하고, MCP 표준 인터페이스로 다른 MCP 클라이언트(예: Claude Code)가 본 시스템을 "증거 읽기 도구"로 안전하게 호출할 수 있습니다. "전자동 적용 금지"와 "실패 케이스도 학습 자산(blocked lesson)으로 보존"이라는 두 원칙을 처음부터 일관되게 지켰습니다.

---

④ 아키텍처 · 흐름도

, HWP .

---

⑤ 예상 결과물

(1) 결과물 형태

Flask 기반 웹 워크스페이스(로컬 127.0.0.1:5000, 7개 탭: Chat/Patch, Goal Brief, Benchmarks, Dream, Memory, Graph, Settings/Health), **CLI** 인터페이스(python main.py --terminal/--web/--index), 31개 미니 프로젝트 벤치마크 러너, **MCP** 서버 **2**종(읽기 전용 + 좁은 범위 artifact-write), 그리고 모든 실행 증거(JSON·Markdown)가 남는 .agent/ 산출물 디렉터리로 구성됩니다.

(2) 핵심 기능 · 사용자

학생에게 제공되는 핵심 가치는 다음과 같습니다: 코드 설명(Chat)으로 모르는 코드를 풀어 설명, 패치 카드로 위험도·자신도·변경 파일을 한눈에 확인, 임시 클론 테스트로 진짜 코드를 보호하면서 검증, 31케이스 벤치마크로 "정말 고치는지" 객관적 증거 제공, Dream 노트로 야간 연구·아이디어 정리, Lesson 승인으로 학생 스스로가 메모리를 만드는 학습 경험. 주요 사용자는 학부 1–4학년 학생, AI 도구를 처음 도입한 코딩 동아리·캡스톤 팀, 그리고 AI 도구 사용을 지도해야 하는 교수·조교입니다.

(3) 테스트 결과

벤치마크 31케이스/7개 언어(C++, Go, HTML, JS, Python, Rust, TS) **fast** 모드 **31/31** 통과(평균 약 42초, 약 0.22달러). 7개 **held-out** 적대적 케이스(다중 파일 원인, 콜백 디스패치, false-positive 억제) **7/7** 통과. 패치 최소화 실험에서 정확도 동일·평균 5.6초 단축. Patch Safety Gate · 임시 클론 분리 · Lesson 승인 흐름은 통합 + e2e 테스트로 검증되어 있습니다.

---

⑥ 의의 · 차별점

(1) 적합성 **— Agentic AI** 부문의 실제 학습 문제 해결

본 출품작은 "AI가 코드를 대신 짜는" 일반적 에이전트가 아니라, 학생이 **AI**를 안전하게 쓰도록 돕는 에이전트형 학습 보조 시스템입니다. Agentic AI의 강점(자동 컨텍스트 수집, 자동 검증, 자동 메모리)가 가져오 되자율성을 의도적으로 제한하여 "사람이 결정한다"는 교육 원칙을 지킵니다.

항목
Cursor / Copilot / Claude Code
C.A.T.T.U.S
주 목적
코딩 생산성 향상
학습 + 안전 + 검증
자동 적용
기본 권장

기본 차단 (Patch Safety Gate)
검증·벤치마크
사용자 책임
임시 클론 자동 실행 + 31케이스 빌트인
메모리·외부 에이전트
모델 컨텍스트 / 직접 실행
사람 **Promote** 시에만 활성 / Mayor Mode + 임시 클론
요약하면 C.A.T.T.U.S는 **"생산성 도구 위에 얹는 안전·학습 계층"** 입니다. 두 종류의 도구는 충돌하지 않으며, 가 때 학생이 가 장사용많할이 배웁니다.

(3) 의의

학습 : 학생이 AI 코드를 그대로 받지 않고, "왜" "가 위험한지"를 보게 됩니다.
안전 : 갸 클론 → 안전 게이트 → 인간 임승시인을 거칩니다.
재현성 : 모든 결정은 .agent/ 하위 JSON·MD가 지도자가 남검아토할 수 있습니다.
확장성 : MCP 표준을 따르므로 다른 LLM·IDE와 연결 가능합니다.

---

⑦ 기장 가 능성 · 후속 계획

(1) 기술·기능 확장 방향

학생 친화적 한국어 설명 강화, 의존 그래프·위험도 히트맵·벤치마크 회귀 추세를 한눈에 보는 학생용 대시보드 확장, 강의 자료(슬라이드·강의록 마크다운)를 reference로 임포트하는 수업용 모드 정식화, 휴대폰에서 결과를 확인하는 모바일 뷰 등을 계획 중입니다.

(2) 적용 범위 · 후속 계획

학과·동아리에서는 캡스톤·졸업작품 지도용으로 사용하고, MCP 호환을 바탕으로 타 대학·고등 SW 교육 도구와 연계할 수 있습니다. 지역 산업 협력(대전·세종·충청권 SW 중소기업 인턴 멘토링)에서 코드 리뷰 보조로 활용하고, 우송대 SW 캡스톤·산학 협력 과제와 연계해 1학기 파일럿을 진행한 뒤 결과를 사례집으로 정리, 후속 SW 교육 대회·해커톤 출품을 검토합니다.

(4) 한계와 향후 개선 방향 **(**정직하게 기재**)**

본 시스템은 다음과 같은 한계가 있습니다.
가 에이전트가 자아율닙니다. 인간 승인 단계가 항상 필요하며, 의도적으로 그렇게 설계되었습니다.
모든 버그를 고칠 수 있는 도구가 아닙니다. 벤치마크 31/7케이스는 통제된 미니 프로젝트이며, 실제 대규모 코드베이스에서는 추가 검증이 필요합니다.
AI 모델 의존성 : Gemini/DeepSeek/Ollama 등 외부 또는 로컬 모델 가용성에 따라 동작이 달라집니다.
PostgreSQL/임베딩**/Ollama**는 선택 사항입니다. 가 모드로도 동작하지만, 일부 고급 기능(메모리·임베딩 검색)은 제한됩니다.
언어 커버리지 : 현재 벤치마크는 7개 언어이며, 그 외 언어에서는 보조 수준이 떨어질 수 있습니다.
보안·라이선스 자동 검토는 부분적입니다. 본 시스템은 위험 경로 차단·검증을 돕지만, 학생이 외부 라이브러리 라이선스를 직접 확인해야 합니다.
이러한 한계를 숨기지 않고 사용자에게 노출하는 것 자체가 본 출품작의 교육 철학입니다.

---

⑧ 지역 협력 방안 **(**선택**)**

우송대 캠퍼스 : SW·AI 관련 학과 수업·동아리·캡스톤 지도에 도입.
대전·세종·충청권 **SW** 기업·기관 : 인턴 학생의 코드 리뷰 보조 도구로 시범 적용. AI 코딩 도구 사용 가이드라인 수립에 기여.
지역 **SW** 교육 커뮤니티 : 오픈 워크숍·세미나에서 "AI 코딩 도구를 어떻게 안전하게 가르칠 것인가" 사례 공유.

---

⑨ 참고 자료

본 출품작이 참고·활용한 주요 도구·라이브러리·문서·아이디어입니다. 가 모델은 학생이 가 진외부자체 API 키로 호출하며, 본 시스템은 그 결과를 안전하게 다루는 계층을 제공합니다.
AI 모델**·API** : Google Gemini API (패치/설명 초안, https://ai.google.dev/), DeepSeek API (리뷰어, https://api-docs.deepseek.com/), Ollama 로컬 LLM 런타임 (https://ollama.com/), Anthropic Claude / Claude Code (Mayor Mode 외부 워커, https://www.anthropic.com/).

핵심 오픈소스 : Flask (https://flask.palletsprojects.com/), PostgreSQL + pgvector (https://github.com/pgvector/pgvector), Tree-sitter (https://tree-sitter.github.io/), pytest (https://docs.pytest.org/), Playwright (https://playwright.dev/).

표준·프로토콜 : Model Context Protocol (https://modelcontextprotocol.io/), MCP Inspector (https://github.com/modelcontextprotocol/inspector).

아이디어·비교 대상 : GitHub Copilot (https://github.com/features/copilot), Cursor IDE (https://www.cursor.com/), Claude Code (Anthropic) — 본 시스템은 이를 하지 않고 으로 위치, SWE-bench (https://www.swebench.com/).

□□ □□□ **(□□ □ □□ □□)** : README.md(□□□ □□ □□, □ 37 KB), AI_Coding_Mentor_Roadmap_Refresh_2026-05-16.md(□□□ □□□·□□□□ □□), .agent/benchmarks/(□□□□□□□), .agent/dreams/·.agent/proposals/·.agent/lessons/(Dream·□□·□□□ □□□ □□ □□).

— □□□□□ □□□□□□ —

□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □
□ □